Embodied AI
Lab Manual for Isaac Lab/Isaac Sim
G1 Stairs Task Setup
August 2026

Prepared by

Mario Ortiz

Table of Contents

1 Objective

The goal of this lab is to create a custom Isaac Lab locomotion task in which the Unitree G1 robot walks forward and climbs a staircase. Unlike previous labs, which focus primarily on modifying the robot and environment configuration, this lab focuses on modifying the terrain generator to create a custom staircase terrain.

In this task, the robot begins on a flat runway and walks toward a staircase. The staircase consists of steps that gradually increase in height. After climbing the stairs, the robot reaches a flat platform at the top.

2 File Structure

```
C:\isaaclab
├── source
│   └── isaaclab_tasks
│       └── isaaclab_tasks
│           └── manager_based
│               └── locomotion
│                   └── velocity
│                       └── config
│                           └── g1
│                               ├── __init__.py
│                               ├── rough_env_cfg.py
│                               ├── flat_env_cfg.py
│                               ├── stair_env_cfg.py
│                               │
│                               └── agents
│                                   ├── __init__.py
│                                   ├── rsl_rl_ppo_cfg.py
│                                   ├── skrl_flat_ppo_cfg.yaml
│                                   └── skrl_rough_ppo_cfg.yaml
│
└── logs
    └── rsl_rl
        └── g1_stairs
```

```
└── YYYY-MM-DD_HH-MM-SS
    ├── model_0.pt
    ├── model_3500.pt
    └── model_6999.pt
```

3 Staircase Environment

The staircase environment provides a structured terrain challenge for evaluating locomotion over changes in elevation. Unlike randomly generated rough terrain, a staircase presents a predictable sequence of obstacles that requires the robot to adjust its gait as the step height changes.

This environment is useful for studying how terrain geometry affects locomotion and for evaluating whether a trained policy can maintain stable forward motion while climbing. It also provides a clear example of how modifying the terrain generator can be used to create new locomotion tasks in Isaac Lab.

The staircase course consists of:

- A flat runway at the beginning of the course
- A series of steps with gradually increasing heights.
- A flat platform at the top of the staircase.

3.1 Create the Stair Environment File

```
cd
C:\isaaclab\source\isaaclab_tasks\isaaclab_tasks\manager_based\locomotion\velocity\config\g1

code stair_env_cfg.py
```

3.2 Import Required Libraries

These imports are needed for terrain creation, reinforcement-learning configuration, and G1 robot setup.

```
import numpy as np

import trimesh

import torch

import math

from isaaclab.terrains import MeshPyramidStairsTerrainCfg, TerrainImporterCfg,
TerrainGeneratorCfg

from isaaclab_assets import G1_MINIMAL_CFG
```

3.3 Create the Custom Stair Terrain Function

The staircase terrain is created using this function:

```
def stair_terrain(difficulty: float, cfg: MeshPyramidStairsTerrainCfg):
```

Isaac Lab terrain functions return a list of terrain meshes and an origin point. The terrain is built using simple box meshes created with trimesh.

3.3.1 Terrain Dimensions

The staircase task defines:

```
runway_length = 3.0
platform_length = 4.0
strip_width = cfg.size[1]
num_steps = int(cfg.size[0] // cfg.step_width)
```

runway_length: flat approach area before the stairs

platform_length: flat area after the stairs

strip_width: width of the staircase strip

num_steps: number of steps based on terrain length and step width

3.3.2 Creating Box Meshes

This helper function

```
def create_box(dims, pos):
    matrix = trimesh.transformations.translation_matrix(pos)
    return trimesh.creation.box(dims, matrix)
```

creates each part of the terrain as a rectangular box. Each box has:

dims = box dimensions

pos = box center position

3.4 Flat Starting Runway

This function adds the runway:

```
meshes_list.append(
    create_box(
```

```
        (runway_length, strip_width, 0.2),
        (runway_length / 2.0, strip_width / 2.0, -0.1),
    )
)
```

This is added to allow the robot to start walking before reaching the first step. The runway is placed slightly below z = 0 so that the top surface is at approximately ground level.

3.5 Create Stairs with Increasing Height

The staircase is created using a for loop:

```
current_total_height = 0.0
for k in range(num_steps):
    step_height = 0.02 + (k / num_steps) * 0.13
    current_total_height += step_height
```

As k increases, step height increases.

Each step is created as a box:

```
current_total_height = 0.0
for k in range(num_steps):
    step_height = 0.02 + (k / num_steps) * 0.13
    current_total_height += step_height
```

The height of each stair box is the total accumulated height, so the staircase rises upward as the robot moves forward.

3.6 Add the Top Platform

After the staircase, a flat platform is also added, giving the robot a flat area to reach after climbing the stairs.

```
top_pos = (
    runway_length + (num_steps * cfg.step_width) + (platform_length / 2.0),
    strip_width / 2.0,
    current_total_height - 0.1,
)

meshes_list.append(
```

```
    create_box(
        (platform_length, strip_width, 0.2),
        top_pos,
    )
)
```

3.7 Setting the Terrain Origin

```
origin = np.array([0.5, strip_width / 2.0, 0.0])
return meshes_list, origin
```

The origin defines where the robot is initially placed relative to the custom terrain.

3.8 Define the G1 Reward Class and Create G1 Stair Environment Class

The reward class is based on the normal G1 rough-terrain rewards:

```
@configclass
class G1Rewards(RewardsCfg):


termination penalty
linear velocity tracking
yaw velocity tracking
feet air time
feet slide penalty
ankle joint limit penalty
hip joint deviation penalty
arm joint deviation penalty
finger joint deviation penalty
torso joint deviation penalty
```

The main environment class is:

```
@configclass
class G1StairEnvCfg(LocomotionVelocityRoughEnvCfg):
```

This means the staircase task will inherit from Isaac Lab's rough-terrain locomotion environment. The major change is that the terrain is replaced with the custom staircase generator.

3.9 Replace the Terrain with the Staircase

Inside __post_init__, the terrain is replaced using:

```
self.scene.terrain = TerrainImporterCfg(
    prim_path="/World/ground",
    terrain_type="generator",
    terrain_generator=TerrainGeneratorCfg(
        seed=42,
        size=(12.0, 1.2),
        border_width=0.0,
        num_rows=1,
        num_cols=1,
        curriculum=False,
        sub_terrains={
            "stair_strip": MeshPyramidStairsTerrainCfg(
                function=stair_terrain,
                proportion=1.0,
                step_width=0.40,
                step_height_range=(0.05, 0.15),
            )
        },
    ),
)
```

Important settings:

- terrain_type=”generator”: use a generated terrain
- size(12.0, 1.2): makes a long and narrow terrain strip
- num_rows=1, num_cols=1: create one staircase course
- curriculum=False: does not vary terrain difficulty
- Step_width=0.40: each step is 0.40m deep

The custom function:

```
function=stair_terrain
```

tells Isaac Lab to use the custom staircase generator instead of the default terrain.

3.10 Run Parent Environment Setup

The environment calls:

```
super().__post_init__()
```

which runs the standard rough-terrain locomotion setup from the parent class.

We will alter many default environment features such as observations, commands, rewards, events, and contact sensors.

3.10.1 Increase Episode Length

We increase episode length as it takes more time to do the complete obstacle.

```
self.episode_length_s = 50.0
self.max_episode_length = int(
    self.episode_length_s / (self.sim.dt * self.decimation)
)
```

3.10.2 Force the Robot to Face the Staircase

The reset yaw is locked to zero:

```
self.scene.height_scanner.prim_path = "{ENV_REGEX_NS}/Robot/torso_link"
```

The starting position is also limited:

```
self.events.reset_base.params["pose_range"]["x"] = (-0.1, 0.1)
self.events.reset_base.params["pose_range"]["y"] = (-0.1, 0.1)
```

This forces the robot to spawn at the beginning of the course and face the stairs. This ensures a controlled environment, so the robot attempts to solve the same task in the same initial point/direction.

3.10.3 Add a Base Height Termination

We add height-based termination if the robot root drops too low. This is necessary for the staircase task because falling off or through the terrain should end the episode.

```
self.terminations.base_height = DoneTerm(
    func=mdp.root_height_below_minimum,
    params={"minimum_height": -0.2},
)
```

3.10.4 Keep G1 Rough-Terrain Reward Adjustments

The code keeps reward settings like the rough G1 environment.

```
self.rewards.lin_vel_z_l2.weight = 0.0
self.rewards.undesired_contacts = None
self.rewards.flat_orientation_l2.weight = -1.0
self.rewards.action_rate_l2.weight = -0.005
self.rewards.dof_acc_l2.weight = -1.25e-7
self.rewards.dof_torques_l2.weight = -1.5e-7
```

These keep the robot focused on locomotion while still penalizing unstable body orientation, excessive action changes, joint acceleration, and torque usage.

3.10.5 Force Forward Motion Toward the Stairs

The command ranges are changed so the robot walks forward only:

```
self.commands.base_velocity.ranges.lin_vel_x = (0.4, 0.6)
self.commands.base_velocity.ranges.lin_vel_y = (0.0, 0.0)
self.commands.base_velocity.ranges.ang_vel_z = (0.0, 0.0)
```

3.10.6 Set the Fall Termination Body

The task uses the G1 torso link for fall termination:

```
self.terminations.base_contact.params["sensor_cfg"].body_names = "torso_link"
```

If the torso contacts the ground or staircase, the episode is terminated.

3.11 The Play Configuration

The play class inherits all staircase setup from G1StairEnvCfg.

It uses multiple environments:

```
self.scene.num_envs = 32
```

It also uses a fixed forward command:

```
self.commands.base_velocity.ranges.lin_vel_x = (0.5, 0.5)
```

3.12 Registering the Stair Task

The task is registered onto the G1's __init__.py file:

PowerShell:

```
cd
C:\isaaclab\source\isaaclab_tasks\isaaclab_tasks\manager_based\locomotion\velocity\config\g1

code __init__.py

Import:

from .stair_env_cfg import G1StairEnvCfg, G1StairEnvCfg_PLAY
```

Register the training task:

```
gym.register(
    id="Isaac-Velocity-Stairs-G1-v0",
    entry_point="isaaclab.envs:ManagerBasedRLEnv",
    disable_env_checker=True,
    kwargs={
        "env_cfg_entry_point": G1StairEnvCfg,
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:G1RoughPPORunnerCfg",
    },
)
```

Register the play task:

```
gym.register(
    id="Isaac-Velocity-Stairs-G1-Play-v0",
    entry_point="isaaclab.envs:ManagerBasedRLEnv",
    disable_env_checker=True,
    kwargs={
        "env_cfg_entry_point": G1StairsEnvCfg_PLAY,
        "rsl_rl_cfg_entry_point": f"{agents.__name__}.rsl_rl_ppo_cfg:G1RoughPPORunnerCfg",
    },
)
```

Add the Stair PPO Runner Configuration

```
code agents\rsl_rl_ppo_cfg.py
```

add:

```
@configclass
class G1StairPPORunnerCfg(G1RoughPPORunnerCfg):
    def __post_init__(self):
```

```
        super().__post_init__()


        self.max_iterations = 7000
        self.experiment_name = "g1_stairs"
        self.num_steps_per_env = 48
        self.algorithm.learning_rate = 5.0e-4
        self.algorithm.desired_kl = 0.008
        self.algorithm.entropy_coef = 0.01
```

3.13 Test the Staircase Code

Verify if the training setup is working properly:

```
cd C:\isaaclab
.\isaaclab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
  --task Isaac-Velocity-Stairs-G1-v0 `
  --num_envs 4 `
  --max_iterations 1
```

Now train the G1. Note that the number of iterations is high due to the complexity and length of the training environment.

```
cd C:\isaaclab
.\isaaclab.bat -p .\scripts\reinforcement_learning\rsl_rl\train.py `
  --task Isaac-Velocity-Stairs-G1-v0 `
  --num_envs 1024 `
  --max_iterations 7000 `
  --headless
```

Play the Trained Stair Policy:

Locate the Stair Run

Note that **$g1StairPath** represents the specific folder in which your model is played in. You will find this name in the folder:

```
\isaacLab\logs\rsl_rl\
```

Initial Checkpoint

```
cd C:\isaaclab
.\isaaclab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
  --task Isaac-Velocity-Stairs-G1-Play-v0 `
  --checkpoint "$g1StairPath\model_0.pt"
```

Middle checkpoint:

```
cd C:\isaaclab
.\isaaclab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
  --task Isaac-Velocity-Stairs-G1-Play-v0 `
  --checkpoint "$g1StairPath\model_1500.pt"
```

Final Checkpoint:

```
cd C:\isaaclab
.\isaaclab.bat -p .\scripts\reinforcement_learning\rsl_rl\play.py `
  --task Isaac-Velocity-Stairs-G1-Play-v0 `
  --checkpoint "$g1StairsPath\model_2999.pt"
```

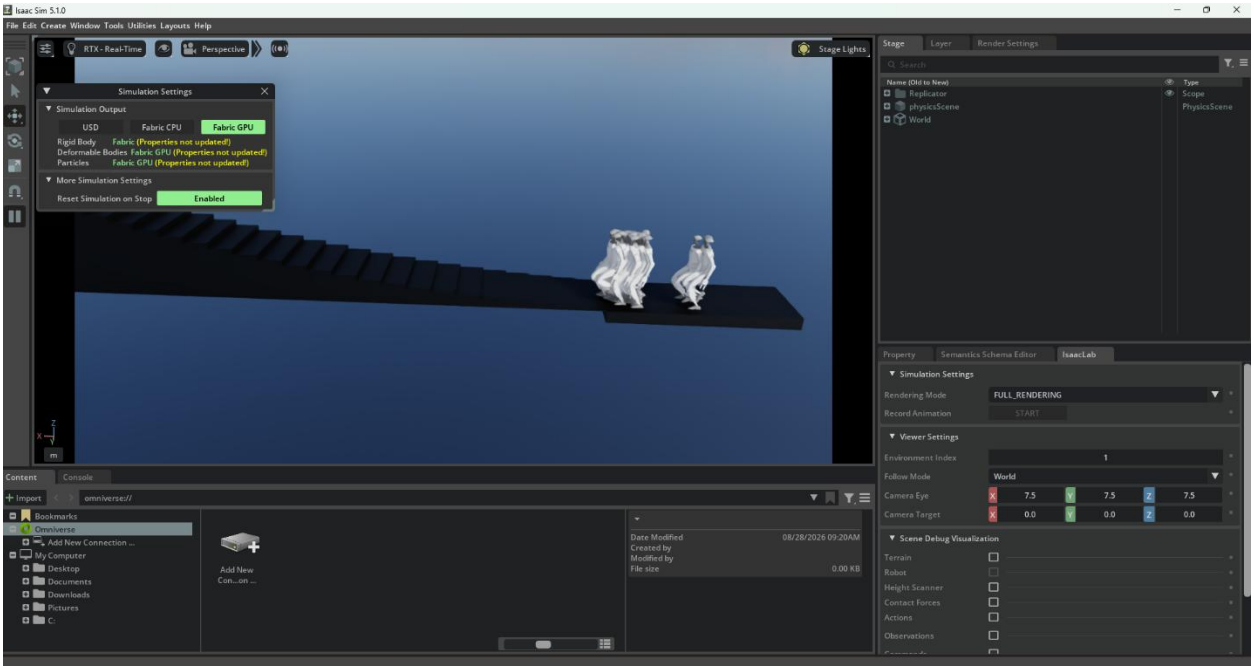

Figure 1: Initial Starting Point of the 32 Unitree G1's spawning in and walking towards the steps

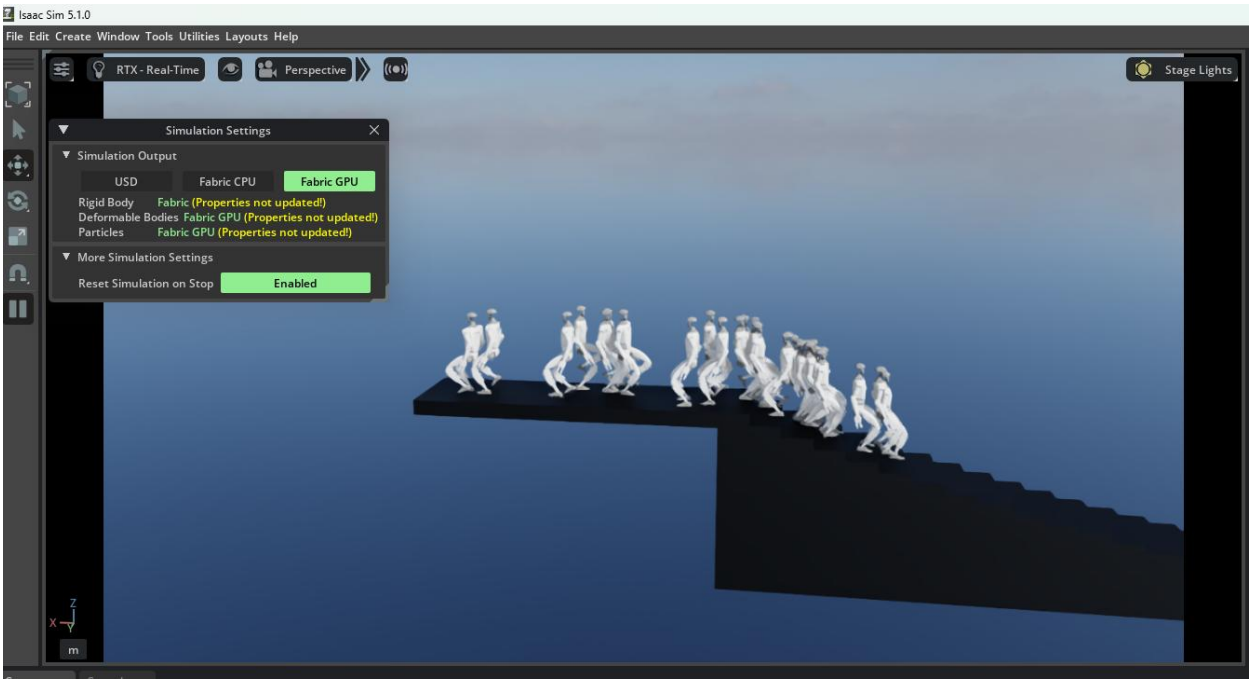

Figure 3: End point of the terrain; you can see that due to the longer episode length, the robots are also trained to turn around and start walking down the steps.

4 References

[1] NVIDIA, “isaaclab.terrains,” Isaac Lab API Documentation.

[2] NVIDIA, “MeshPyramidStairsTerrainCfg,” Isaac Lab API Documentation.

[3] NVIDIA, “Creating a Manager-Based RL Environment,” Isaac Lab Documentation.

[4] NVIDIA, “Registering an Environment,” Isaac Lab Documentation.

[5] Trimesh, “trimesh.creation.box,” Trimesh Documentation.

[6] Trimesh, “trimesh.transformations,” Trimesh Documentation.

[7] NumPy, “NumPy Documentation.”

[8] NVIDIA, “isaaclab_rl,” Isaac Lab RL API Documentation.

[9] RSL-RL, “Configuration,” RSL-RL Documentation.